Appendix

A Proof of lemmas and theories

A.1 Lemma3.1

Proof. The total number of points in the dataset is N. These points are partitioned into K_1K_2 disjoint sub-clusters, $C_1, C_2, \ldots$, such that $N = \sum_i |C_i|$.

The array P_i is a sequence of prefix sums based on the counts of points of C_i falling into fixed grid regions. Since the counts are non-negative, the values in P_i are non-decreasing integers.

The maximum value in the prefix sum array P_i is the final element, which represents the total count of all points of C_i across all grid regions. Therefore,

$$\max(P_i) = |C_i|$$

Since the values in P_i are non-negative integers that range from 0 up to a maximum of $|C_i|$, the number of distinct values in the array, $card(P_i)$, can be at most $|C_i| + 1$ (if the array contains 0 and all integers up to $|C_i|$). A conservative upper bound that is sufficient for our argument is:

$$card(P_i) \leq |C_i|$$

Now, summing this inequality over all K_1K_2 sub-clusters:

$$\sum_{i=1}^{K_1K_2} card(P_i) \leq \sum_{i=1}^{K_1K_2} |C_i|$$

Since the sub-clusters C_i are disjoint and partition the dataset $\mathcal{U}$, their total size is N:

$$\sum_{i=1}^{K_1K_2} |C_i| = N$$

Substituting this total back into the inequality yields the result:

$$\sum_{i=1}^{K_1K_2} card(P_i) \leq N$$

Intuitively, $card(P_i)$ measures the complexity of cluster C_i's distribution relative to the grid structure. The total measure of complexity across all clusters is bounded by the total number of points, N, because the number of distinct counts in P_i can be no greater than the total number of points belonging to C_i. □

A.2 Lemma3.2

Proof (by contradiction). Assume, for the sake of contradiction, that there exist $\mathbf{x}, \mathbf{y} \in \mathbb{Z}^l$ such that $\forall i,\ x_i \leq y_i$, but $morton(\mathbf{x}) > morton(\mathbf{y})$.

The Morton code is constructed by interleaving the binary digits of each coordinate, i.e.,

$$morton(\mathbf{x}) = interleave(x_{l-1}, \ldots, x_0).$$

Let b be the most significant bit position at which the interleaved binary representations of $morton(\mathbf{x})$ and $morton(\mathbf{y})$ differ. This bit must correspond to some dimension j such that

$$x_j^{(b)} \neq y_j^{(b)}, \quad \text{and for all } b' > b,\ x^{(b')} = y^{(b')}.$$

Since $morton(\mathbf{x}) > morton(\mathbf{y})$, we have $x_j^{(b)} = 1$ and $y_j^{(b)} = 0$. However, this implies $x_j > y_j$, which contradicts our assumption that $x_j \leq y_j$.

Therefore, the assumption is false, and we conclude that

$$morton(\mathbf{x}) \leq morton(\mathbf{y}).$$

□

A.3 Theory1

Proof. **Lower bound.** By definition of the compressed prefix sum array, each non-compressible substructure requires at least one storage unit. Therefore, the space lower bound $\Theta\left(\frac{G K_1 K_2}{S[l]}\right)$ directly follows from the structural definition and is omitted here for brevity.

Upper bound. Let R denote the number of rows in the compressed array. According to the hierarchical and block organization of the data structure, the number of rows satisfies

$$R = \Theta\left(\frac{G}{S[l]}\right).$$

Consider an arbitrary row, and let S_{row} denote the total number of *sumNode* elements stored in this row. By **Lemma 3.2** and by the definition of the compressed prefix sum array (which explicitly avoids storing duplicate or redundant prefix values), each original data item can contribute to at most a constant number of distinct *sumNode*s within a single row. Hence, there exists a constant $c > 0$ such that

$$S_{\text{row}} \leq c \cdot N,$$

implying that each row occupies $O(N)$ space. In the general case, the row must still encode the cumulative contribution of all N original elements, leading to an asymptotically tight bound

$$S_{\text{row}} = \Theta(N).$$

Combining the per-row complexity with the number of rows yields the overall upper bound:

$$\text{Space} = R \cdot S_{\text{row}} = \Theta\left(\frac{G}{S[l]}\right) \cdot \Theta(N) = \Theta\left(\frac{N G}{S[l]}\right).$$

Together with the lower bound derived from the definition, this completes the proof. □

A.4 Theory2

Proof. **Setup and notation.** Let R denote the number of rows in the compressed array at layer l. By the hierarchical/block organisation of the structure we have

$$R = \Theta\left(\frac{G}{S[l]}\right) = \Theta\left(\frac{G}{\sqrt{G}}\right) = \Theta(\sqrt{G}).$$

From the previous theorem (upper bound with general $S[l]$) we know that the worst-case space per stored row is $\Theta(N)$; hence the worst-case total storage (without accounting for row deduplication) is $\Theta(N) \cdot R = \Theta(N\sqrt{G})$. We next show that, in expectation, row-level deduplication and the mandatory-first-entry constraint reduce the expected stored fraction of rows by a factor bounded above by the stated $\max(\cdot, \cdot)$ term, yielding the claimed expected bound.

Probability that a row is identical to its predecessor. Consider two consecutive logical rows in the structure. If the later row is *entirely zero* (i.e. all its G positions are zero after projection onto the considered row's support), then its prefix-sum vector equals

that of the previous row and therefore it need not be stored explicitly. Hence a sufficient condition for a row to be identical to its predecessor is that the row is all-zero.

Let M denote the number of non-zero positions (or an available upper bound thereof) in the global array; sampling $s = \sqrt{G}$ positions per row (as in the layer/block arrangement) yields the exact combinatorial probability that a given row is entirely zero:

$$p_{\text{zero}} \;=\; \frac{\binom{G-M}{s}}{\binom{G}{s}} \;=\; \prod_{i=0}^{s-1} \frac{G-M-i}{G-i}.$$

Using the standard inequality $1 - x \le e^{-x}$ and $s = \sqrt{G}$, we obtain the exponential upper bound

$$p_{\text{zero}} \;\le\; \exp\Big(-\frac{sM}{G}\Big) \;=\; \exp\Big(-\frac{M}{\sqrt{G}}\Big).$$

Recalling the relation between M and N used in the earlier lemma (or setting M to the problem-specific upper bound $N/(C\,S_c)$), we further bound

$$p_{\text{zero}} \;\le\; \exp\Big(-\frac{N}{C\,S_c\,\sqrt{G}}\Big).$$

Thus the probability that a row is *not* all-zero (and hence, under the sufficient condition considered, could require explicit storage) is at most $1 - p_{\text{zero}}$.

Lower bound on the fraction of rows that must be stored. The previous paragraph gives a probabilistic reason for row equality via the all-zero event, but it does not capture deterministic constraints that force storage even when rows are not all-zero. In particular, each secondary table (secondary index or per-block subordinate structure) must store its first entry; these mandatory entries impose a non-vanishing lower bound on the fraction of rows that cannot be eliminated by deduplication. Concretely, standard combinatorial/entropy arguments on the distribution of first entries across M possible values imply that at least a fraction on the order of $M^{-1/(2(l-1))}$ of rows (up to constants) must be distinct in expectation (see main text for the per-secondary-table occupancy argument). We therefore obtain a deterministic-type lower bound $f_{\min}$ on the expected stored fraction of rows:

$$f_{\min} \;\ge\; M^{-\frac{1}{2(l-1)}}.$$

Combining the two effects. Let X be the number of rows actually stored (random variable). For $i \ge 2$ row i is stored only if it is not identical to row $i-1$ (and the first row is stored according to the mandatory-first-entry rule). Taking expectations, using independence assumptions between rows as in the earlier lemma (rows are formed from independent projections and their all-zero events are independent), and combining the probabilistic and deterministic constraints above, we obtain

$$\mathbb{E}[X] \;\le\; 1 + (R-1)\cdot\max\big(1 - p_{\text{zero}},\, f_{\min}\big).$$

Multiplying by the per-row storage cost $\Theta(N)$ yields

$$\mathbb{E}[\text{Space}] \;=\; \Theta(N)\cdot\mathbb{E}[X] \;=\; \Theta(N)\left(1 + (R-1)\max\Big(1 - p_{\text{zero}},\, f_{\min}\Big)\right).$$

Substituting $R = \Theta(\sqrt{G})$ and discarding lower-order additive constants gives

$$\mathbb{E}[\text{Space}] \;=\; \Theta\Big(N\sqrt{G}\Big)\cdot\max\Big(1 - p_{\text{zero}},\, M^{-\frac{1}{2(l-1)}}\Big).$$

Replace $1 - p_{\text{zero}}$ by its exponential bound. Using the exponential bound for p_{zero} derived above we have $1 - p_{\text{zero}} \le 1 - \exp(-N/(C\,S_c\sqrt{G}))$. For the regime of interest (large parameters typical in our applications) we can upper-bound $1 - p_{\text{zero}}$ by a constant multiple of $\exp(-N/(C\,S_c\sqrt{G}))$ (this is the standard simplification when the exponential term is small). Thus

$$\mathbb{E}[\text{Space}] \;=\; O\left(N\sqrt{G}\cdot\max\left(1 - \exp\Big(-\frac{N}{CS_c\sqrt{G}}\Big),\; M^{-\frac{1}{2(l-1)}}\right)\right).$$

Why the little-o notation. The stated bound is written with $o(\cdot)$ (rather than $\Theta(\cdot)$) because the first row (and possibly a small constant number of initial rows) contains strictly fewer *sumNode*s than a typical later row; consequently, when $R > 1$ the per-row $\Theta(N)$ worst-case is not attained uniformly across all rows, and the asymptotic leading constant is strictly smaller than that of the worst-case product $N\sqrt{G}$. Therefore the expected total storage lies in $o\big(N\sqrt{G}\cdot\max(\cdot,\cdot)\big)$ rather than $\Theta(\cdot)$. Combining the above estimates completes the proof. □

A.5 Theory3

Proof. Recall that a single JumpIn call is invoked to traverse one contiguous interval (segment) of the underlying one-dimensional ordering (Morton/Z-order) that intersects the query filter. Hence the total number of JumpIn calls for a given range query is upper-bounded by the number of disjoint contiguous Morton-intervals into which the query region is decomposed.

It is a classic result that encoding a range (rectangular) query using Morton order yields $\Theta(\mathcal{A}(\mathcal{R}))$ such contiguous segments, where $\mathcal{A}(\mathcal{R})$ is the surface area of the query region (see e.g. [**? ?**]). We therefore denote by $m = \Theta(\mathcal{A}(\mathcal{R}))$ the number of contiguous Morton segments generated for the query when no additional splits are applied.

Now consider the effect of applying s splits to the data structure. Each split can at most increase the total number of contiguous segments intersecting the query by one: a split that falls inside a previously contiguous segment may split that segment into two, thereby adding at most one extra segment; a split that falls outside the query or exactly on a segment boundary does not increase the number of segments. Hence after applying s splits the total number of segments is at most $m + s$. Consequently, the number of JumpIn calls is upper-bounded by

$$m + s \;=\; \Theta\big(\mathcal{A}(\mathcal{R})\big) + s \;=\; O\big(\mathcal{A}(\mathcal{R}) + s\big).$$

Conclusion. Putting the two bounds together, we obtain

$$\#\text{JumpIn} \;=\; \Theta\big(\mathcal{A}(\mathcal{R}) + s\big),$$

as required. □